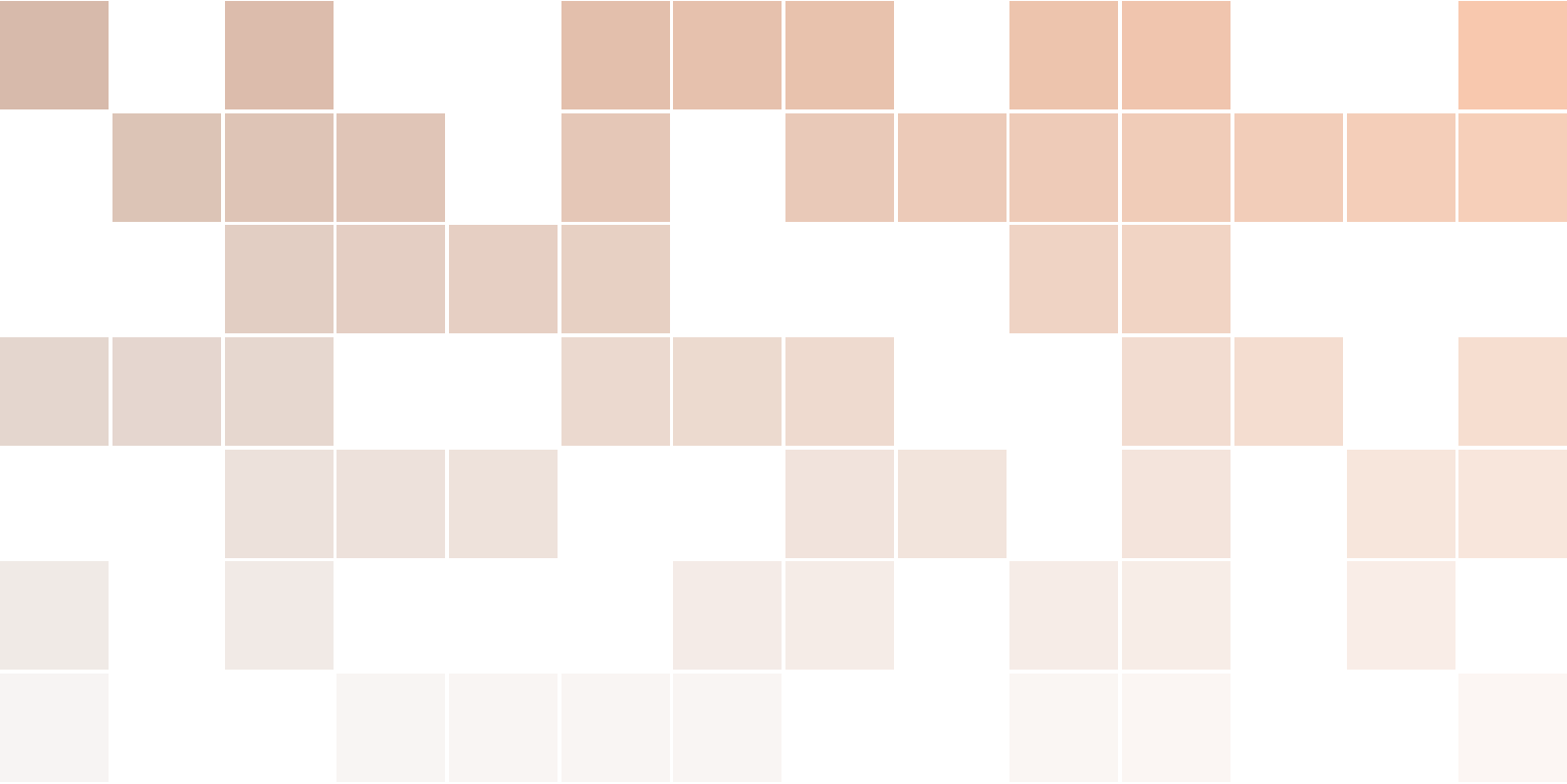
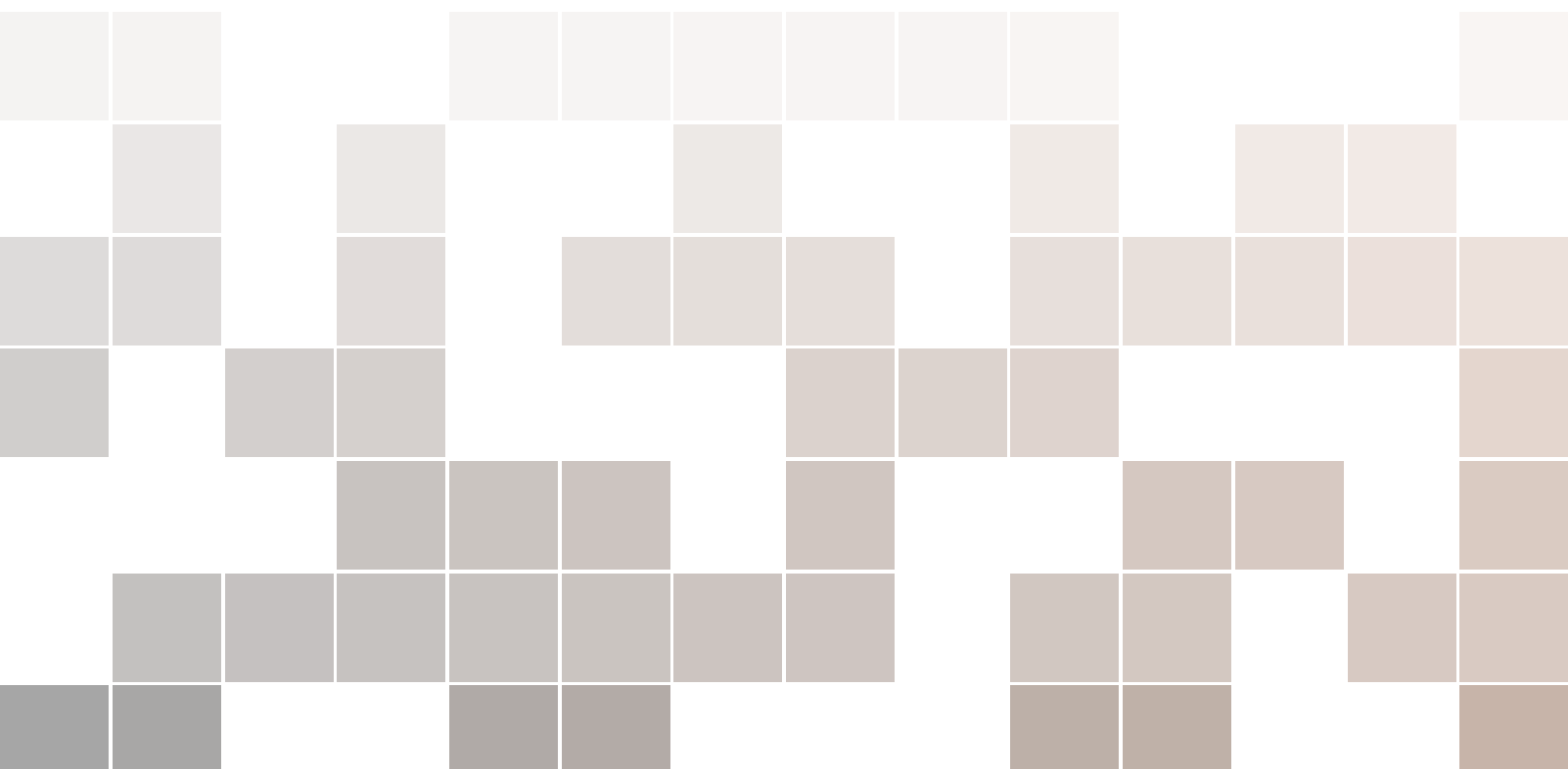




Manual de códigos - Procesamiento de audio, video y señales fisiológicas

Python

Nathaly Ramón L.



Contents

I	Procesamiento de audio	
1	Limpieza de audio	7
1.1	Conversión a formato adecuado	7
1.1.1	Variables de entrada	7
1.1.2	Formato de Salida	8
1.2	Cortar video	8
1.2.1	Variables de entrada	8
1.3	Proceso de limpieza	9
1.3.1	Fundamentos y referencias al paper de Liu, 2018	9
1.3.2	Pipeline de limpieza	10
1.3.3	Variables de entrada	10
1.3.4	Recomendaciones de ajuste ante problemas	11
2	Modificaciones para modelo de IA	13
2.1	Adición ruido tráfico	13
2.1.1	Variables de entrada	14
2.2	Adición ruido blanco gaussiano	14
2.2.1	Variables de entrada	14
2.3	Pitch Shifting	15
2.3.1	Variables de entrada	15
2.4	RMS	16
2.4.1	Detectar valores RMS bajos	16
2.4.2	Encontrar valores estadísticos de RMS	16
2.4.3	Normalización por RMS	17

2.5	MFCC	17
2.5.1	Variables de entrada	17

II **Procesamiento de video**

3	Preparación de videos sin sincronización	23
3.1	Redimensión de video	23
3.2	Corte de video	23
4	Preparación de videos con sincronización	25
4.1	Conversión de HEVC a H264	25
4.2	Variables de entrada	26
4.3	Corte de video con sincronismo	26
4.4	Redimensión de cortes de video	26
5	Modificaciones para modelo de IA	29
5.1	Girar video hacia izquierda o derecha	29
5.1.1	Variables de entrada	29
5.2	Modificar brillo de videos	30
5.2.1	Variables de entrada	30
5.3	Efecto Blur	30
5.3.1	Variables de entrada	31
5.4	Adición de ruido AWGN	31
5.4.1	Variables de entrada	31

III **Procesamiento de señales fisiológicas**

6	Señales de EMOTIBIT	35
6.1	Limpiar señales	35
6.2	Variables de entrada	36
7	Código para abrir archivos generados	37
7.1	Variables de entrada	38
	Bibliography	39
	Books	39



Procesamiento de audio

1	Limpieza de audio	7
1.1	Conversión a formato adecuado	
1.2	Cortar video	
1.3	Proceso de limpieza	
2	Modificaciones para modelo de IA ...	13
2.1	Adición ruido tráfico	
2.2	Adición ruido blanco gaussiano	
2.3	Pitch Shifting	
2.4	RMS	
2.5	MFCC	

1. Limpieza de audio

1.1 Conversión a formato adecuado

Este script está diseñado para la conversión masiva (batch) de archivos de vídeo en formato .mp4 a archivos de audio en formato .wav. Fue desarrollado para extraer la pista de audio de múltiples grabaciones de video y estandarizarlas en un formato de audio sin compresión.

El nombre del código implementado es:

Código 1.1 conversor_mp4_wav.py

El script sigue un flujo secuencial:

1. Lee las rutas de configuración definidas por el usuario (FFmpeg, carpeta origen y carpeta destino).
2. Crea automáticamente la carpeta de salida si no existe.
3. Recorre todos los archivos con extensión .mp4 dentro de la carpeta de origen.
4. Para cada archivo, construye un comando del sistema que invoca a FFmpeg con los siguientes parámetros clave:
 - vn → omite el track de video (solo audio).
 - acodec pcm_s16le → convierte el audio a PCM de 16 bits (sin pérdida).
 - ar 44100 → fija la frecuencia de muestreo en 44.1 kHz.
 - y → sobrescribe el archivo de salida si ya existe.
5. Ejecuta el comando mediante subprocess y registra si la conversión fue exitosa o falló.
6. Al finalizar, muestra un resumen con el total de archivos procesados, exitosos y con error.

1.1.1 Variables de entrada

ruta_ffmpeg: Ruta absoluta al ejecutable de FFmpeg en el sistema local.

carpeta_videos: Ruta a la carpeta donde están los archivos .mp4 fuente

carpeta_salida: Ruta donde se guardarán los archivos .wav resultantes. Si no existe, se crea automáticamente.

1.1.2 Formato de Salida

- **Extensión:** .wav
- **Códec de audio:** PCM signed 16-bit little-endian (pcm_s16le).
- **Frecuencia de muestreo:** 44 100 Hz.
- **Canales:** Se respeta la cantidad de canales del archivo original (mono o estéreo).
- **Regla de nombrado:** [NombreOriginal]_01.wav. El sufijo _01 se añade para evitar sobrescrituras accidentales con archivos preexistentes.

1.2 Cortar video

Script diseñado para segmentar automáticamente archivos de audio .wav en partes más pequeñas, utilizando como referencia un pitido de frecuencia fija (1198 Hz). Es usado para dividir grabaciones largas donde cada bloque está delimitado por un tono de sincronización. El script procesa todos los .wav de una carpeta y guarda los recortes en subcarpetas individuales.

El script implementado es:

Código 1.2 cut_video.py

1. Carga del audio con librosa para análisis espectral.
2. Detección del pitido:
 - Calcula la transformada de Fourier (STFT) y extrae la energía en la banda de frecuencia especificada (1198 Hz).
 - Normaliza la energía y aplica un umbral de sensibilidad para identificar los frames donde el pitido está presente.
3. Filtrado de pitidos: Convierte los frames a tiempos en segundos y agrupa aquellos que estén separados por más de 2 segundos (evita detecciones múltiples de un mismo pitido prolongado).
4. Recorte del audio usando pydub:
 - Para cada pitido detectado, establece el punto de inicio como tiempo_pitido + 300 ms (para eliminar el propio pitido en el clip).
 - El final del segmento se define como:
 - Si hay un pitido siguiente: tiempo_siguiente - 50 ms (corta justo antes del siguiente pitido).
 - Si es el último pitido: inicio + 4590 ms (duración fija de 4.59 segundos para el último bloque).
5. Guardado: Cada segmento se exporta como un archivo .wav independiente dentro de una subcarpeta con el nombre del archivo original (dentro de la carpeta de salida master).

Nota: El valor de umbral_sensibilidad puede requerir ajuste experimental. Si el script detecta pitidos donde no los hay (falsos positivos) o no detecta algunos pitidos reales, modifica este parámetro, se sube para ser más estricto o baja para ser más sensible.

Nota: El valor de frecuencia_pitido se ha determinado luego de usar la herramienta Audacity en donde se pudo ver el espectro del audio previamente y determinar el valor de frecuencia del pitido.

1.2.1 Variables de entrada

carpeta_entrada: Carpeta donde están los archivos .wav originales a procesar.

carpeta_salida_master: Carpeta raíz donde se crearán las subcarpetas con los recortes.

frecuencia_pitido: Frecuencia del tono a detectar (por defecto 1198 Hz).

umbral_sensibilidad: Valor entre 0 y 1 que regula la sensibilidad de detección. Ajustar si el pitido se confunde con voz u otros ruidos.

Observación: En el mapeo masivo de la data subida, en la mayoría de los casos hubo problemas con la emoción Enojo, ajustar el umbral acorde a lo necesario.

1.3 Proceso de limpieza

Script de limpieza y acondicionamiento de audio para reconocimiento de emociones en voz. Procesa archivos .wav previamente segmentados y aplica un pipeline de 11 etapas de filtrado, reducción de ruido, supresión de música, detección de actividad vocal (VAD) y normalización. El objetivo es obtener señales de voz limpias, estandarizadas y libres de interferencias, listas para la extracción de características (como MFCC o GFCC) y su posterior clasificación emocional. Está basado en los fundamentos del paper de Liu (2018) sobre GFCC para reconocimiento de emociones. Soporta procesamiento masivo de toda una carpeta de recortes, manteniendo la estructura de subcarpetas.

El script es el siguiente:

Código 1.3 limpiar_audio.py

1.3.1 Fundamentos y referencias al paper de Liu, 2018

El paper de Liu (2018) propone los Gammatone Frequency Cepstral Coefficients (GFCC) como representación superior a los MFCC para la clasificación de emociones. Sin embargo, para que dichas características sean efectivas, la señal de entrada debe estar libre de ruido, interferencias y artefactos que puedan enmascarar los patrones emocionales. Este script implementa un preprocesamiento robusto que:

- Elimina el zumbido de red eléctrica (60 Hz) y las bajas frecuencias (DC, vibraciones) que no son relevantes para la voz.
- Filtra ultrasonidos (>10 kHz) que no aportan información emocional y pueden saturar el análisis.
- Aplica sustracción espectral por bandas con diferentes factores de atenuación (alfa) para proteger la banda donde se concentran los armónicos de la voz (especialmente la femenina en estados de enojo).
- Implementa un filtro de Wiener género-adaptivo que evita el efecto "metálico" en voces femeninas al establecer un piso de ganancia mínimo en la banda vocal (165–4000 Hz).
- Incluye un supresor de música de fondo basado en la estabilidad temporal del espectro, ya que la música (a diferencia de la voz emocional) presenta cambios lentos en su magnitud espectral.
- Utiliza un VAD espectral con dos criterios (SFM y centroide) para silenciar fragmentos sin voz, protegiendo los ataques bruscos del enojo con un contexto temporal amplio.
- Finaliza con un noise gate, pre-énfasis y normalización RMS (comentado en el script) para nivelar la energía.

Nota: Las etapas que aparecen comentadas en el script, son necesarias únicamente durante el proceso de generación de los coeficientes MFCC. Aunque forman parte del pipeline de preprocesamiento y limpieza de la señal, no son indispensables para la ejecución del procedimiento principal. Por esta razón, dichas etapas se implementarán en un script independiente, dedicado exclusivamente al cálculo de los coeficientes MFCC.

1.3.2 Pipeline de limpieza

En la tabla 1.1 se indican las etapas de limpieza por las cuales pasarán los archivos con el objetivo de obtener audios que no tengan factores ajenos a las características de la voz acorde a su emoción.

Etapas	Proceso	Descripción técnica
1. Carga y conversión	Lectura con pydub, conversión a mono, 44.1 kHz, float32.	Se estandariza el formato de entrada para que todos los filtros operen con la misma frecuencia de muestreo y canal único.
2. Notch 60 Hz	Filtro notch IIR de orden 2 con $Q=30$.	Elimina la interferencia de la red eléctrica (zumbido) sin afectar al contenido espectral de la voz.
3. HPF 80 Hz	Filtro Butterworth pasa-altos de orden 4.	Atenúa frecuencias por debajo de 80 Hz (DC, vibraciones, ruido de baja frecuencia) que no son relevantes para la voz humana.
4. LPF 10 kHz	Filtro Butterworth pasa-bajos de orden 4.	Elimina ultrasonidos y ruido de alta frecuencia que pueden distorsionar el análisis.
5. Sustracción espectral por bandas	Estimación del perfil de ruido a partir de los primeros 50 ms. Cada banda tiene un factor alfa diferente	Elimina el efecto hiss (siseo). La banda media tiene el alfa más bajo para proteger los armónicos de la voz femenina en estados de enojo (que concentran energía en esa región).
6. Wiener género-adaptivo	Filtro de Wiener con SNR mínima garantizada (0.3) y un piso de ganancia (0.35) en la banda 165-4000 Hz.	Suprime el ruido residual, pero evita la atenuación excesiva en la banda vocal (especialmente crítica para voces femeninas), previniendo el efecto metálico de la voz. El suavizado temporal (ventana de 5 frames) evita variaciones perceptibles.
7. Supresor de música	Detecta música de fondo por estabilidad temporal del espectro: calcula la correlación de Pearson entre ventanas consecutivas de magnitudes espectrales. Si la correlación supera un umbral (0.87) durante 4 frames seguidos, se etiqueta ese bin como música.	La música tiene cambios espectrales lentos, mientras que la voz emocional varía rápidamente. Se aplica una atenuación de 18 dB en bins de música, pero solo 8 dB en la banda vocal protegida para no dañar la voz.
8. VAD espectral	Dos criterios: SFM (flatness measure) < 0.50 , o centroide espectral entre 100 y 4500 Hz. Se usan ventanas de 100 ms para no recortar ataques de voz.	Identifica frames con voz (actividad vocal). Los frames sin voz se atenúan con ganancia 0.02. El umbral SFM más alto (0.50 vs 0.45) protege la voz femenina enojada, cuyo espectro es más plano.
9. Noise Gate	En ventanas de 20 ms calcula RMS y compara con umbral de 45 dBFS y release de 60 ms.	Silencia completamente los segmentos con energía muy baja, eliminando residuos de ruido que puedan quedar tras las etapas anteriores.

Table 1.1: Etapas del pipeline de preprocesamiento de audio.

El orden de las etapas es crítico: primero los filtros lineales, luego la reducción de ruido, después la supresión de música y el VAD, y al final los ajustes de ganancia.

1.3.3 Variables de entrada

ruta_bin_ffmpeg: Ruta a la carpeta que contiene ffmpeg.exe y ffprobe.exe.

carpeta_recortes: Carpeta donde están los recortes .wav

carpeta_final: Carpeta de salida donde se guardarán los audios limpios (se crea automáticamente).

1.3.4 Recomendaciones de ajuste ante problemas

Para determinados problemas es posible ajustar algunos parámetros del script. Se indica la tabla 1.2 con el problema y acción sugerida.

Problema	Solución sugerida
Voz femenina enojada suena metálica	Subir SS_ALFA_MEDIA (ej. 1.4) o bajar WIENER_SNR_FLOOR.
Música persiste en el fondo	Subir MUSICA_ESTABILIDAD_UMBRAL (ej. 0.85).
Voz suena recortada o entrecortada	Bajar VAD_UMBRAL_SFM, subir VAD_CONTEXTO_MS.
Zumbido residual	Subir GATE_UMBRAL_DB (ej. -40).

Table 1.2: Posibles problemas durante el procesamiento y ajustes sugeridos.

Estos ajustes deben ser modificados, de ser el caso, con cuidado ya que cada parámetro modificado podría tener efectos en otras características de la voz. Los valores definidos en código se establecieron en base a pruebas y usando como referencia valores del paper.

2. Modificaciones para modelo de IA

2.1 Adición ruido tráfico

Script diseñado para añadir ruido de calle (tráfico) y reverberación ambiental a audios de voz previamente limpios. Su objetivo es generar versiones "ruidosas" de los recortes de voz para entrenar modelos robustos ante entornos acústicos reales. Se somete a adición directa de ruido del tipo autos y tráfico y se ejecuta la convolución con un IR de un espacio exterior de zona urbana.

El código en cuestión es:

Código 2.1 ruido_calle_add.py

El script sigue este flujo secuencial:

1. Configuración del motor: Establece las rutas de FFmpeg para que pydub pueda manejar archivos .mp3 y .wav.
2. Carga de efectos base (una sola vez al inicio):
 - Tráfico: Archivo .mp3 largo con sonido de coches y bocinas.
 - Respuesta al impulso (IR): Archivo .wav (test_outdoor.wav) que contiene la "firma acústica" de un entorno exterior.
3. Convolución con la IR:
 - Convierte el audio de voz a un array NumPy.
 - Aplica la convolución (fftconvolve) entre la señal de voz y la IR. Al convolucionar la voz con esta IR, se simula el efecto de reverberación, eco y filtrado espacial característico de ese entorno, haciendo que la voz suene como si realmente se hubiera grabado en la calle.
4. Procesamiento de la voz: La señal convolucionada se pasa por un filtro pasa-altos de 250 Hz (para eliminar graves excesivos que pueda añadir la IR) y se ajusta su ganancia (VOL_VOZ_POST_CONV).
5. Selección de ruido de tráfico: Se toma un segmento aleatorio del archivo de tráfico que tenga exactamente la misma duración que el audio de voz procesado.
6. Mezcla final: Se superpone (overlay) el tráfico sobre la voz, con el nivel de volumen del tráfico

ajustable (NIVEL_TRAFICO_DB). El resultado final se convierte a estéreo (2 canales).

7. Guardado: Se exporta el audio mezclado en una estructura de carpetas igual al de entrada.

2.1.1 Variables de entrada

ruta_bin_ffmpeg: Ruta ala carpeta que contiene ffmpeg.exe y ffprobe.exe.

carpeta_recortes: Carpeta donde están los recortes .wav

carpeta_final: Carpeta de salida donde se guardarán los audios con ruido.

ruta_trafico: Ruta al archivo de audio .mp3 con sonido de tráfico ambiente.

ruta_ir_calle: Ruta al archivo .wav que contiene la Respuesta al Impulso (IR) del entorno exterior.

NIVEL_TRAFICO_DB: Nivel de volumen del ruido de tráfico en dB relativo a la voz.

VOL_VOZ_POST_CONV: Ganancia aplicada a la voz después de la convolución.

2.2 Adición ruido blanco gaussiano

Script diseñado para añadir ruido blanco gaussiano (AWGN) a audios de voz. Su objetivo es generar versiones degradadas con diferentes relaciones señal-ruido (SNR) para entrenar modelos de reconocimiento de emociones. Procesa masivamente toda una carpeta de recortes y guarda los resultados con un sufijo personalizable, manteniendo la estructura de subcarpetas pero renombradas.

El código desarrollado es:

Código 2.2 ruido_blanco_gaussiano_add.py

El flujo de procesamiento por cada archivo es el siguiente:

1. Carga del audio:
 - Lee el archivo .wav, lo convierte a mono y a 44.1 kHz.
2. Conversión a NumPy:
 - Obtiene un array de valores normalizados en el rango [-1, 1].
3. Cálculo de la potencia de la señal:
 - Se calcula la potencia media (RMS) de la voz: $\text{potencia_senal} = \text{mean}(\text{señal}^2)$.
4. Cálculo de la potencia del ruido según el SNR deseado:
 - Se aplica la fórmula:

$$\text{potencia_ruido} = \frac{\text{potencia_senal}}{10^{(\text{SNR_deseado}/10)}}$$
 - Donde SNR_deseado se ingresa en decibelios (ej. 15 dB).
5. Generación del ruido gaussiano:
 - Se genera una muestra aleatoria de una distribución normal (Gaussiana) con media 0 y desviación estándar $\sqrt{\text{potencia_ruido}}$.
 - El tamaño del ruido es idéntico al de la señal de voz.
6. Mezcla:
 - Se suma directamente el ruido a la señal de voz (adicción en el dominio del tiempo).
7. Guardado:
 - Se convierte el array resultante de vuelta a AudioSegment y se exporta como .wav con el sufijo configurado, replicando la estructura de carpetas pero añadiendo el sufijo a cada subcarpeta.

2.2.1 Variables de entrada

ruta_bin_ffmpeg: Ruta ala carpeta que contiene ffmpeg.exe y ffprobe.exe.

carpeta_recortes: Carpeta donde están los recortes .wav

carpeta_final: Carpeta de salida donde se guardarán los audios con ruido.

SNR_DESEADO_DB: Relación señal-ruido objetivo en decibelios. Ajuste crítico: valores bajos (ej. 5 dB) producen mucho ruido, valores altos (ej. 25 dB) apenas se nota. Por defecto 15 dB.

2.3 Pitch Shifting

Script diseñado para modificar el tono (pitch) de audios de voz, agudizándolos o graveándolos según un número de semitonos configurable. Su propósito es generar versiones con diferente altura tonal (por ejemplo, simulando voces más agudas o más graves) para aumentar la diversidad del dataset. Procesa masivamente toda una carpeta de recortes y guarda los resultados con un sufijo específico, replicando la estructura de subcarpetas con el sufijo añadido.

El script indicado es el siguiente:

Código 2.3 pitch_shifting_add.py

El flujo de procesamiento por cada archivo es el siguiente:

1. Carga del audio:
 - Lee el archivo .wav, lo convierte a mono y a 44.1 kHz.
2. Cálculo del factor de cambio:
 - Convierte los semitonos a un factor de escala:
$$\text{factor} = 2^{(\text{semitonos}/12.0)}$$
 - En la escala musical temperada, un semitono equivale a un factor de $2^{1/12}$ en frecuencia. Por tanto, subir 2 semitonos multiplica todas las frecuencias por $2^{2/12} \approx 1.122$.
3. Cambio de tono por velocidad de muestreo:
 - Crea una versión del audio donde el frame_rate se multiplica por el factor calculado. Esto hace que el audio se reproduzca más rápido (tono más agudo) o más lento (tono más grave).
 - Inmediatamente después, se re-muestrea el audio de vuelta a la frecuencia de trabajo original (44.1 kHz).
 - Este proceso produce un cambio de tono sin alterar la duración del audio.
4. Guardado:
 - Se exporta el audio modificado con el sufijo configurado, replicando la estructura de carpetas pero añadiendo el sufijo a cada subcarpeta.

2.3.1 Variables de entrada

ruta_bin_ffmpeg: Ruta a la carpeta que contiene ffmpeg.exe y ffprobe.exe.

carpeta_recortes: Carpeta donde están los recortes .wav

carpeta_final: Carpeta de salida donde se guardarán los audios con pitch modificado.

PITCH_SHIFT_SEMITONES: Número de semitonos para desplazar el tono. Valores positivos (ej. 2.0, 3.5) agudizan la voz; valores negativos (ej. -2.0, -4.0) la gravean.

Nota: Para generar la data modificada se ejecutó el script con valores de 2 y -2 para archivos mas agudos y graves, respectivamente.

Nota: La técnica utilizada es un método estándar y computacionalmente ligero para modificar el tono sin alterar la duración. Matemáticamente, equivale a un reescalado en frecuencia del espectro: todas las componentes de frecuencia se multiplican por el mismo factor. Esto produce variaciones realistas en la voz (voces más agudas o graves) sin introducir artefactos significativos.

2.4 RMS

Este conjunto de tres scripts conforma un flujo de procesamiento secuencial cuyo objetivo es homogeneizar el nivel de energía (RMS) de todos los audios de un dataset, eliminando previamente aquellos que sean silencios o tengan una energía excepcionalmente baja.

La normalización por RMS es fundamental para que los modelos de reconocimiento de emociones no se vean afectados por diferencias de volumen entre grabaciones, micrófonos o sesiones, y para que el peso energético de cada emoción (p.ej. el enojo, que tiene picos altos pero silencios, vs. la tristeza, más sostenida) sea comparable.

2.4.1 Detectar valores RMS bajos

Escanea recursivamente una carpeta de audios .wav, calcula el RMS de cada uno y lista (o elimina) aquellos cuyo RMS está por debajo de un umbral configurable. Estos archivos suelen ser silencios, grabaciones fallidas, o fragmentos con voz casi inaudible que solo introducirían ruido y valores erróneos en el objetivo de establecer un umbral de RMS. El código implementado para esto es:

Código 2.4 detectar_RMS_bajo.py

Este script sigue el siguiente esquema:

1. Lee cada .wav y calcula RMS, pico, duración
2. Ordena los resultados por RMS ascendente y muestra un reporte con estadísticas globales y la lista de sospechosos.
3. En modo seguro (MODO_SEGURO = True) solo muestra los archivos que serían eliminados; en modo ejecución (False) los borra físicamente.

Nota: Primero enlistar los archivos sospechosos y posterior a ello ejecutar el script en MODO_SEGURO = False para eliminarlos

2.4.2 Encontrar valores estadísticos de RMS

Recorre la misma carpeta (ya depurada) y calcula el RMS de cada archivo .wav (promediando a mono si es necesario). Al final imprime cuatro estadísticos: mínimo, máximo, promedio y mediana de todos los RMS. El código es:

Código 2.5 find_RMS.py

El objetivo de este script es encontrar el valor de RMS promedio que se usará como RMS_OBJETIVO en el script de normalización. Este valor representa el nivel de energía típico del conjunto y es un punto de referencia adecuado para igualar todos los audios sin saturar ni atenuar excesivamente.

2.4.3 Normalización por RMS

En esta parte, se aplica una ganancia escalar a cada archivo de audio para que su RMS (energía promedio) sea exactamente igual a un valor objetivo (RMS_OBJETIVO). Procesa todos los .wav de una carpeta y guarda los resultados en una nueva estructura de carpetas, añadiendo un sufijo configurable a cada nombre de archivo y de carpeta.

Variables de entrada

carpeta_entrada: Ruta de la carpeta que contiene los audios ya depurados

carpeta_salida: Ruta donde se guardarán los audios normalizados. Se crea automáticamente.

RMS_OBJETIVO: Valor de RMS deseado. Se recomienda usar el RMS promedio obtenido con `find_RMS.py`

2.5 MFCC

Script diseñado para extraer características acústicas MFCC (Mel Frequency Cepstral Coefficients) de audios de voz previamente limpiados, siguiendo el pipeline propuesto por Liu (2018) para reconocimiento de emociones en habla. Genera una representación vectorial por cada frame de audio, incluyendo contexto temporal (ventana de ± 9 frames), y guarda los resultados en formato comprimido .npz junto con la etiqueta de emoción inferida del nombre del archivo.

El código es:

Código 2.6 MFCC_liu.py

El script aplica secuencialmente el pipeline estándar de MFCC indicado en Liu,2018

1. Pre-énfasis ($\alpha=0.97$) para compensar la atenuación de altas frecuencias.
2. Framing: Ventanas de 25 ms con salto de 10 ms.
3. Ventaneo Hamming.
4. FFT de 2048 puntos.
5. Banco de filtros Mel: Con 40 bandas, rango 80–8000 Hz.
6. Compresión logarítmica.
7. DCT: Para obtener 13 coeficientes MFCC base.
8. Deltas (Δ) y dobles deltas ($\Delta\Delta$): Vector de 39 coeficientes por frame.
9. CMN (Cepstral Mean Normalization): Restando la media de cada coeficiente (opcional).
10. Representación contextual:
 - Para cada frame t , concatena los 39 coeficientes de los frames $t - 9$ a $t + 9$, generando un vector de 741 dimensiones por frame (39×19).

2.5.1 Variables de entrada

carpeta_limpios: Ruta de la carpeta que contiene los audios limpios.

carpeta_mfcc: Carpeta de salida donde se guardarán los archivos .npz con las características.

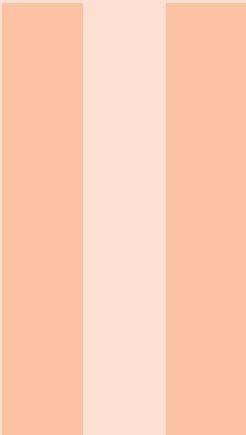
MAPA_EMOCIONES: Diccionario que asocia códigos (segundo segmento del nombre) con etiquetas de emoción.

Los archivos de salida son del tipo .npz. Estos archivos contienen:

- frames: array de tipo float32 con forma (n_frames, 741), donde cada fila es el vector contextual de 741 dimensiones (39 coeficientes $\times$ 19 frames).
- emocion: array de tipo str con una única etiqueta (ej. "enojo").

Adjunto al repositorio de códigos se encuentra un script de ayuda para abrir y visualizar un fragmento de estos archivos .npz. El script es cuestión es:

Código 2.7 abrir_npz.py



Procesamiento de video

3 Preparación de videos sin sincronización 23

- 3.1 Redimensión de video
- 3.2 Corte de video

4 Preparación de videos con sincronización 25

- 4.1 Conversión de HEVC a H264
- 4.2 Variables de entrada
- 4.3 Corte de video con sincronismo
- 4.4 Redimensión de cortes de video

5 Modificaciones para modelo de IA . . . 29

- 5.1 Girar video hacia izquierda o derecha
- 5.2 Modificar brillo de videos
- 5.3 Efecto Blur
- 5.4 Adición de ruido AWGN

En el repositorio de datos, los videos provenían de fuentes y condiciones de grabación distintas, lo que obligó a establecer dos pipelines de procesamiento diferenciados. El factor determinante para elegir una u otra ruta fue la calidad y disponibilidad del audio asociado a cada archivo de video

1. Flujo sin sincronización (audio deficiente o inexistente)

Cuando el audio del video era de baja calidad, presentaba ruido excesivo, o simplemente no contenía información sincronizada útil. En estos casos, el audio no era confiable para segmentar los clips, por lo que se prescindió de la sincronización temporal.

Primero se redimensiona el video largo. Se procesa el archivo completo, se detecta el rostro del sujeto, se recorta y se escala a una resolución cuadrada de 224×224 píxeles.

Segundo se segmenta en clips. Sobre el video ya redimensionado, se realiza un corte temporal fijo (por ejemplo, clips de 5 segundos de duración) sin necesidad de alineación con eventos de audio.

2. Flujo con sincronización (audio utilizable y con pitidos de referencia)

Cuando el audio del video era limpio y contenía los pitidos de sincronización a 1198 Hz (el mismo estándar usado para segmentar los audios en `cut_audio.py`). En estos casos, el audio sí era confiable para alinear los cortes con los estímulos o ensayos correspondientes.

Primero se corta el video en clips sincronizados: Utilizando el audio asociado al video, se detectan los pitidos y se generan clips de video que coinciden exactamente con los segmentos de audio ya extraídos

Segundo se redimensionan los clips. Una vez que se tienen los clips cortos, cada uno de ellos pasa por el proceso de detección de rostro, recorte y redimensionamiento a 224×224 píxeles.

3. Preparación de videos sin sincronización

3.1 Redimensión de video

Script diseñado para procesar videos largos (grabaciones de rostro) y generar una versión redimensionada y centrada en el rostro del sujeto, con salida en formato cuadrado de 224×224 píxeles. El código es:

Código 3.1 redimensionar_video.py

1. Detección del rostro:
 - Intenta localizar el rostro en varios frames (por defecto en los frames 25, 50, 75, 100, 125).
 - Usa el clasificador Haar de OpenCV (haarcascade_frontalface_default.xml).
 - Si detecta un rostro, calcula un cuadro delimitador ampliado (2.2× el tamaño del rostro) y centrado en él.
 - Si no lo detecta en ningún intento, recurre al recorte central del video (cuadrado más grande posible).
2. Redimensionado y codificación:
 - Lee el video frame a frame desde el principio.
 - Recorta cada frame usando el cuadro calculado y lo redimensiona a 224×224.
 - Envía los frames redimensionados directamente a FFmpeg, que los codifica con el códec libx264, preset veryfast y calidad CRF=15 (alta calidad perceptual).
 - Se preserva la frecuencia de fotogramas original del video.
3. Salida:
 - Guarda el video procesado en la carpeta de destino con el sufijo configurado.

3.2 Corte de video

Script diseñado para segmentar videos ya redimensionados (salida de redimensionar_video.py) en fragmentos de duración fija (5 segundos). Es el segundo paso del flujo sin sincronización, donde los videos largos ya han sido procesados y centrados en el rostro, y ahora se dividen en clips cortos

para su posterior análisis o etiquetado. El script utiliza FFmpeg en modo -c copy (stream copy), lo que evita recodificar el video y preserva la calidad original. El código es:

Código 3.2 cortar_video_redimensionado.py

1. Lectura de la duración total del video usando ffprobe
2. Creación de una carpeta de destino específica para ese video
3. Bucle de corte:
 - Desde tiempo_actual = 0.0, se calcula la duración del siguiente fragmento
 - Si el tiempo restante es menor a DURACION_MINIMA_FINAL (2 s), se descarta y se termina el bucle.
4. Guardado: Todos los clips de un mismo video se guardan dentro de una subcarpeta con el nombre del video original.

4. Preparación de videos con sincronización

4.1 Conversión de HEVC a H264

Script diseñado para convertir videos en formato HEVC/H.265 a H.264 de forma masiva, ya que OpenCV y muchas bibliotecas de procesamiento de video no son compatibles de forma nativa con HEVC. Este script es un paso previo al procesamiento de video (tanto en el flujo con sincronización como sin sincronización), asegurando que todos los archivos estén en un códec ampliamente compatible (H.264).

El código es el siguiente:

Código 4.1 HEVC_to_H264.py

El script procesa cada video siguiendo este flujo:

1. Verificación de compatibilidad:
 - Comprueba que FFMpeg y FFprobe estén disponibles en el sistema.
2. Verificación de unidad de almacenamiento:
 - Espera a que las unidades de entrada/salida estén accesibles (útil para discos de red o unidades externas).
3. Detección del códec de video:
 - Usa ffprobe para identificar el códec del archivo (hevc, h265, h264, u otro).
4. Decisión de acción:
 - Si el códec es HEVC/H.265: se recodifica a H.264 usando libx264 con preset ultrafast, calidad CRF=18 y -c:a copy (el audio se mantiene sin cambios).
 - Si el códec ya es H.264 (u otro compatible): se copia el archivo directamente a la carpeta de salida sin recodificar (ahorro de tiempo).
5. Reducción opcional de resolución:
 - Si ANCHO_MAXIMO está definido (ej. 640), escala el video manteniendo la relación de aspecto, reduciendo el lado mayor a ese valor. Esto acelera drásticamente el proce-

samiento posterior.

6. Reanudación automática:

- Si un archivo de salida ya existe, se omite para evitar reprocesar en caso de interrupciones.

4.2 Variables de entrada

ruta_bin_ffmpeg: Ruta absoluta al ejecutable de FFmpeg en el sistema local.

carpeta_entrada: Carpeta que contiene los videos originales (en HEVC u otros formatos).

carpeta_salida: Carpeta donde se guardarán los videos convertidos o copiados.

ANCHO_MAXIMO: Límite de píxeles para el lado mayor del video. 640 es un buen balance para detección de rostro; None mantiene resolución original.

4.3 Corte de video con sincronismo

Script diseñado para segmentar videos largos en clips sincronizados a partir de pitidos de referencia (1198 Hz) presentes en la pista de audio del video. Es el primer paso del flujo con sincronización, donde la calidad del audio es buena y permite alinear los cortes de video con los segmentos de audio previamente extraídos. El script es:

Código 4.2 cortar_gopro.py

1. Extracción de audio temporal:
 - Usa FFmpeg para extraer la pista de audio del video y guardarla como un archivo .wav en la carpeta temporal del sistema.
2. Detección de pitidos:
 - Carga el audio temporal con librosa y aplica el mismo algoritmo que en cut_audio.py:
3. Corte del video para cada pitido detectado:
 - Calcula el punto de inicio = tiempo_pitido + 300 ms (para eliminar el propio pitido).
 - Calcula el final como:
 - Si hay siguiente pitido: tiempo_siguiente - 50 ms.
 - Si es el último: inicio + 4.59 s (duración fija).
 - Usa un comando FFmpeg con seek híbrido (-ss antes y después de -i) para lograr cortes rápidos y precisos.
 - Codifica el segmento con libx264, preset veryfast y calidad CRF=18 (alta calidad).
 - No conserva el audio (-an).
4. Guardado:
 - Todos los clips de un mismo video se guardan en una subcarpeta con el nombre del video original, dentro de carpeta_salida_master.

4.4 Redimensión de cortes de video

Script diseñado para redimensionar y centrar en el rostro los clips de video previamente cortados mediante sincronización por pitidos. Es el segundo paso del flujo con sincronización, donde los clips ya están segmentados temporalmente y ahora se les aplica el procesamiento visual (recorte y escalado a 224×224). Para optimizar el rendimiento, el script detecta el rostro una sola vez por subcarpeta (usando el primer clip) y reutiliza las coordenadas de recorte para todos los clips de esa carpeta, asumiendo que los demás clips de la subcarpeta corresponden a un mismo video pero cortado. Esto evita repetir la detección en cada clip y acelera drásticamente el procesamiento masivo.

El script es:

Código 4.3 redimensionar_cortado_gopro.py

El proceso de este script es el siguiente:

1. Agrupación por subcarpetas:
 - Escanea recursivamente la carpeta de entrada y agrupa los videos según la subcarpeta donde se encuentran. Cada grupo se procesa como una unidad.
2. Detección de rostro (una sola vez por grupo):
 - Toma el primer clip del grupo.
 - Intenta detectar el rostro usando el clasificador Haar de OpenCV en varios frames (por defecto en los frames 15, 30, 45, 60, 75).
 - Si detecta un rostro, calcula un cuadro delimitador ampliado (2.2× el tamaño del rostro) y centrado en él.
 - Si no lo detecta, recurre al recorte central (cuadrado más grande posible) como respaldo.
3. Codificación de todos los clips del grupo:
 - Para cada clip, usa las coordenadas de recorte ya calculadas (sin volver a detectar).
 - Lee el video frame a frame, recorta según esas coordenadas, redimensiona a 224×224 y envía los frames a FFmpeg.
 - Guarda el video redimensionado en la carpeta de salida, replicando la estructura de subcarpetas y añadiendo el sufijo configurado.

5. Modificaciones para modelo de IA

5.1 Girar video hacia izquierda o derecha

Script diseñado para rotar videos 90 grados hacia la izquierda o hacia la derecha. El código es el siguiente:

Código 5.1 girar_video.py

El esquema a seguir es:

1. Configuración de la dirección:
 - Según la variable DIRECCION ("izquierda" o "derecha"), selecciona el valor correspondiente del filtro transpose de FFmpeg:
 - 1 → 90° horario (derecha).
 - 2 → 90° antihorario (izquierda).
2. Construcción de rutas:
 - Replica la estructura de subcarpetas de carpeta_entrada en carpeta_salida, añadiendo un sufijo configurable al nombre del archivo.
3. Ejecución de FFmpeg:
 - Aplica el filtro de rotación manteniendo el códec de video (libx264, preset veryfast, CRF=18) y conservando el audio sin recodificar (-c:a copy).

5.1.1 Variables de entrada

ruta_ffmpeg: Ruta absoluta al ejecutable de FFmpeg en el sistema local.

carpeta_entrada: Carpeta que contiene los videos a rotar.

carpeta_salida: Carpeta donde se guardarán los videos rotados.

DIRECCION: Dirección de giro: "izquierda" (90° antihorario) o "derecha" (90° horario).

5.2 Modificar brillo de videos

Script diseñado para modificar el brillo (exposición) de videos de forma masiva, subiendo o bajando la luminosidad de todos los cuadros mediante un factor multiplicativo. Es una herramienta de data augmentation que permite generar versiones con diferentes condiciones de iluminación para entrenar modelos de IA más robustos ante variaciones de luz.

El script es:

Código 5.2 brillo_video.py

El esquema a seguir es:

1. El script procesa todos los videos de una carpeta (recorriendo subcarpetas) y aplica el filtro `colorchannelmixer` de FFmpeg.
2. El filtro multiplica cada canal de color (rojo, verde, azul) por el factor `NIVEL_BRILLO` especificado.
 - Valores:
 - 1.0 = sin cambios.
 - > 1.0 (ej. 1.3) = aumenta el brillo (imagen más clara).
 - < 1.0 (ej. 0.7) = disminuye el brillo (imagen más oscura).
3. La operación es equivalente a una escala lineal de los valores de píxel, similar al ajuste de exposición en fotografía.
4. Se conserva el formato `yuv420p` para garantizar compatibilidad con reproductores y `frameworks`.
5. El audio se descarta (`-an`), ya que en este contexto solo interesa la modificación visual para data augmentation.

Nota: Para tener efectos visibles se usó un factor de 0.7 para bajar el brillo y 1.2 para subir brillo.

5.2.1 Variables de entrada

ruta_ffmpeg: Ruta absoluta al ejecutable de FFmpeg en el sistema local.

carpeta_entrada: Carpeta que contiene los videos a modificar.

carpeta_salida: Carpeta donde se guardarán los videos con brillo ajustado.

NIVEL_BRILLO: Factor multiplicativo para el brillo. Rango recomendado: 0.7 (más oscuro) a 1.3 (más claro).

5.3 Efecto Blur

Script diseñado para aplicar desenfoque (blur) gaussiano a videos de forma masiva, simulando falta de enfoque o baja resolución. Es una herramienta de data augmentation que permite generar versiones con diferentes niveles de nitidez. El código es:

Código 5.3 blurring_video.py

El flujo del código es el siguiente:

1. El script procesa todos los videos de una carpeta (recorriendo subcarpetas) y aplica el filtro `gblur` de FFmpeg.

2. El filtro aplica un desenfoque gaussiano a cada cuadro, utilizando una desviación estándar (sigma) configurable.
 - Valores típicos de sigma:
 - 0 = sin cambios.
 - 0.5-1.0 = desenfoque sutil.
 - 1.5-2.5 = desenfoque moderado (rango recomendado para augmentation facial).
 - >3.0 = pérdida excesiva de detalle.
3. La operación se realiza en el dominio espacial, suavizando los bordes y reduciendo detalles.
4. El audio se descarta (-an), ya que solo interesa la modificación visual.

5.3.1 Variables de entrada

ruta_ffmpeg: Ruta absoluta al ejecutable de FFmpeg en el sistema local.

carpeta_entrada: Carpeta que contiene los videos a modificar.

carpeta_salida: Carpeta donde se guardarán los videos con desenfoque.

NIVEL_BLUR: Desviación estándar (sigma) del filtro gaussiano. Rango recomendado: 0.5–3.0.

Nota: El valor de NIVEL_BLUR usado para generar la data es 1.5

5.4 Adición de ruido AWGN

Script diseñado para añadir ruido blanco gaussiano (AWGN) a videos de forma masiva, simulando condiciones de grabación con poca luz o sensores de cámara de baja calidad.

El script desarrollado es:

Código 5.4 ruido_video.py

El esquema a seguir es:

1. El filtro `noise=all={intensidad}:allf=t` añade ruido blanco gaussiano a todos los canales de color (alls) con distribución temporal (`allf=t`).
2. Valores típicos de intensidad (escala 0–100):
 - 0 = sin cambios.
 - 5-10 = ruido sutil (simula sensor de gama baja o poca luz leve).
 - 10-20 = ruido moderado (condiciones de poca luz notorias).
 - >25 = degradación excesiva del detalle facial .
3. El ruido se añade en el dominio espacial, afectando la textura de la imagen.
4. Se utiliza formato yuv420p para compatibilidad.
5. El audio se descarta (-an), ya que solo interesa la modificación visual.

5.4.1 Variables de entrada

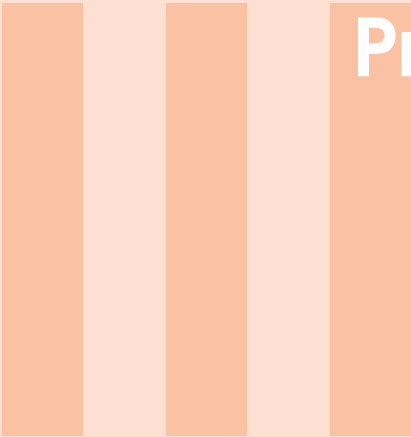
ruta_ffmpeg: Ruta absoluta al ejecutable de FFmpeg en el sistema local.

carpeta_entrada: Carpeta que contiene los videos a modificar.

carpeta_salida: Carpeta donde se guardarán los videos con ruido.

NIVEL_RUIDO: Intensidad del ruido (escala 0–100). Rango recomendado: 5–25.

Nota: El valor usado en este script fue de 20



Procesamiento de señales fisiológicas

6	Señales de EMOTIBIT	35
6.1	Limpiar señales	
6.2	Variables de entrada	
7	Código para abrir archivos generados	37
7.1	Variables de entrada	
	Bibliography	39
	Books	

6. Señales de EMOTIBIT

6.1 Limpiar señales

Script diseñado para limpiar y filtrar señales fisiológicas provenientes del dispositivo Emotibit (EDA y PPG). Toma archivos CSV crudos generados por el dispositivo, extrae las señales de interés (actividad electrodérmica fásica y tónica, PPG en verde, infrarrojo y rojo), aplica filtros digitales para eliminar ruido y artefactos, y genera dos archivos CSV separados por cada archivo de entrada: uno para EDA y otro para PPG, cada uno con su propia frecuencia de muestreo nativa (15 Hz y 25 Hz respectivamente), preservando la sincronización temporal original.

El script desarrollado es:

Código 6.1 Emotibit_filtrado.py

El flujo de procesamiento por cada archivo CSV es el siguiente:

1. Parsing robusto del CSV crudo:
 - Lee el archivo fila por fila, ya que el campo DataLength (columna 3) es variable (cada paquete puede tener 1, 2, 3 o más valores).
 - Extrae solo los tags de interés: EA (EDA fásica), EL (EDA tónica), PG (PPG verde), PI (PPG infrarrojo), PR (PPG rojo).
 - Almacena los paquetes con sus timestamps y valores.
2. Expansión de paquetes:
 - Cada paquete puede contener múltiples muestras; se expande asignando un timestamp a cada muestra, interpolando dentro del paquete usando la frecuencia de muestreo estimada.
 - El timestamp del paquete corresponde a la última muestra; las anteriores se retrocalculan.
3. Estimación de la frecuencia de muestreo real:
 - Se calcula $fs = \text{número_total_muestras} / \text{duración_segundos}$ para cada señal, obteniendo

valores típicos de ~ 15 Hz para EDA y ~ 25 Hz para PPG.

4. Filtrado digital (Butterworth):
 - EDA: filtro pasa-bajas con $\text{cutoff}=1.0$ Hz para la fásica (EA) y $\text{cutoff}=0.05$ Hz para la tónica (EL).
 - PPG: filtro pasa-banda $0.7\text{--}3.5$ Hz para las tres longitudes de onda (PG, PI, PR), aislando la banda de frecuencia cardíaca.
 - Se usa `filtfilt` para fase cero.
5. Generación de DataFrames:
 - Cada señal filtrada se guarda en un DataFrame con columnas `Time_Sec` (tiempo en segundos desde el inicio del archivo), `Value` (crudo), `Clean_Value` (filtrado).
6. Fusión por `merge_asof`:
 - Para EDA: se toma el DataFrame de la fásica (EA) como base y se añade la tónica (EL) mediante `merge_asof` (búsqueda del vecino más cercano en tiempo).
 - Para PPG: se toma el verde (PG) como base y se añaden infrarrojo (PI) y rojo (PR).
7. Guardado:
 - Cada archivo de salida incluye una cabecera con comentarios indicando la fs estimada para cada columna.
 - Se generan dos CSV por archivo de entrada, con sufijos configurados (`_01` para EDA, `_02` para PPG).
8. Opcional:
 - Si `GENERAR_GRAFICAS = True`, abre una ventana por archivo mostrando la comparación crudo vs. filtrado para verificar visualmente.

6.2 Variables de entrada

carpeta_datos: Carpeta donde están los archivos CSV crudos de Emotibit.

carpeta_salida: Carpeta donde se guardarán los CSV procesados (se crea automáticamente).

GENERAR_GRAFICAS: Si es `True`, muestra gráficas de verificación por cada archivo (bloquea la ejecución hasta cerrar la ventana).

Las frecuencias de muestreo de EDA y PPG son diferentes (15 Hz vs 25 Hz), por lo que se generan archivos separados en lugar de forzar un re-muestreo común.

7. Código para abrir archivos generados

Script de visualización diseñado para abrir y graficar los archivos CSV limpios generados por `Emotibit_filtrado.py`. Permite inspeccionar visualmente las señales filtradas de EDA y PPG, mostrando las trazas de cada canal en subplots separados con sus respectivas frecuencias de muestreo (leídas desde los comentarios en la cabecera del CSV).

El código es el siguiente:

Código 7.1 `abrir_emotibit.py`

El flujo de procesamiento es el siguiente:

1. Lectura de rutas:
 - Si se pasan dos argumentos por línea de comandos, los usa como rutas a los archivos EDA y PPG.
 - En caso contrario, usa las rutas por defecto definidas al inicio del script.
2. Parsing de metadata:
 - Lee el archivo CSV línea por línea, extrayendo los comentarios con formato: `# fs_NombreColumna_Hz=valor`.
 - Almacena la frecuencia de muestreo estimada para cada columna de señal.
3. Carga de datos:
 - Utiliza `pd.read_csv` con `comment="#"` para omitir las líneas de metadata y cargar solo los datos numéricos.
4. Graficado:
 - Para cada columna de señal (todas excepto `Time_Sec`), crea un subplot con la señal en función del tiempo.
 - Muestra en el título la frecuencia de muestreo correspondiente si está disponible.
 - Las gráficas se abren en ventanas independientes (una para EDA y otra para PPG) y bloquean la ejecución hasta que el usuario las cierre.

7.1 Variables de entrada

ruta_eda_default: Ruta al archivo CSV de EDA generado por Emotibit *filtrado.py*

ruta_ppg_default: Ruta al archivo CSV de PPG generado por Emotibit *filtrado.py*

A close-up photograph of a red rose, showing the intricate details of its petals and the green sepals at the base. The rose is the central focus of the image, with its vibrant red color contrasting against the blurred background.

Repositorio en Github - Scripts implementados

Link

Link al repositorio Procesamiento de Data